

TASK 2 – API Integration & JSON Handling

Project Title:

Weather Information Finder using Python API

1. Objective

The objective of this project is to learn API integration and JSON handling using Python. The project fetches real-time weather data from an external API and displays important weather information based on user input.

2. Technologies Used

- Python
- Requests Library
- OpenWeatherMap API

3. Features

- Fetches weather data using API
- Parses JSON responses
- Searches weather by city name
- Displays temperature, humidity, and weather condition
- Uses try-except error handling
- Handles invalid city names and API errors

4. Code Explanation

The program takes a city name as input from the user and sends an HTTP request to the OpenWeatherMap API using the Requests library. The API returns weather information in JSON format. The program extracts important weather details such as temperature, humidity, weather description, and wind speed from the JSON response and displays them to the user. Proper exception handling is implemented using try-except blocks.

5. Source Code

```
import requests

# Your API key from OpenWeatherMap
API_KEY = "96fe4e5b45bf87c217596e65e1f5075d"

# Base URL of the API
BASE_URL = "https://api.openweathermap.org/data/2.5/weather"

def get_weather(city):
    """
    Fetch weather data from API
    """
    # Parameters sent to API
    params = {
        "q": city,
        "appid": API_KEY,
        "units": "metric"
    }
    try:
        # Sending GET request to API
        response = requests.get(BASE_URL, params=params)

        # Convert JSON response into dictionary
        data = response.json()

        # Handle API errors
        if response.status_code != 200:
            print("API Error:", data.get("message"))
            return None
        return data
    except requests.exceptions.ConnectionError:
        print("Connection Error! Check internet.")
    except requests.exceptions.Timeout:
        print("Request Timed Out!")
    except requests.exceptions.RequestException as e:
        print("Something went wrong:", e)
    return None
```

```

def display_weather(data):
    """
    Display important weather information
    """
    # Extracting data from JSON response
    city = data["name"]
    country = data["sys"]["country"]
    temperature = data["main"]["temp"]
    humidity = data["main"]["humidity"]
    weather = data["weather"][0]["description"]
    wind_speed = data["wind"]["speed"]

    print("\n===== WEATHER REPORT =====")
    print(f"City: {city}, {country}")
    print(f"Temperature: {temperature}°C")
    print(f"Humidity: {humidity}%")
    print(f"Weather: {weather}")
    print(f"Wind Speed: {wind_speed} m/s")
    print("=====")

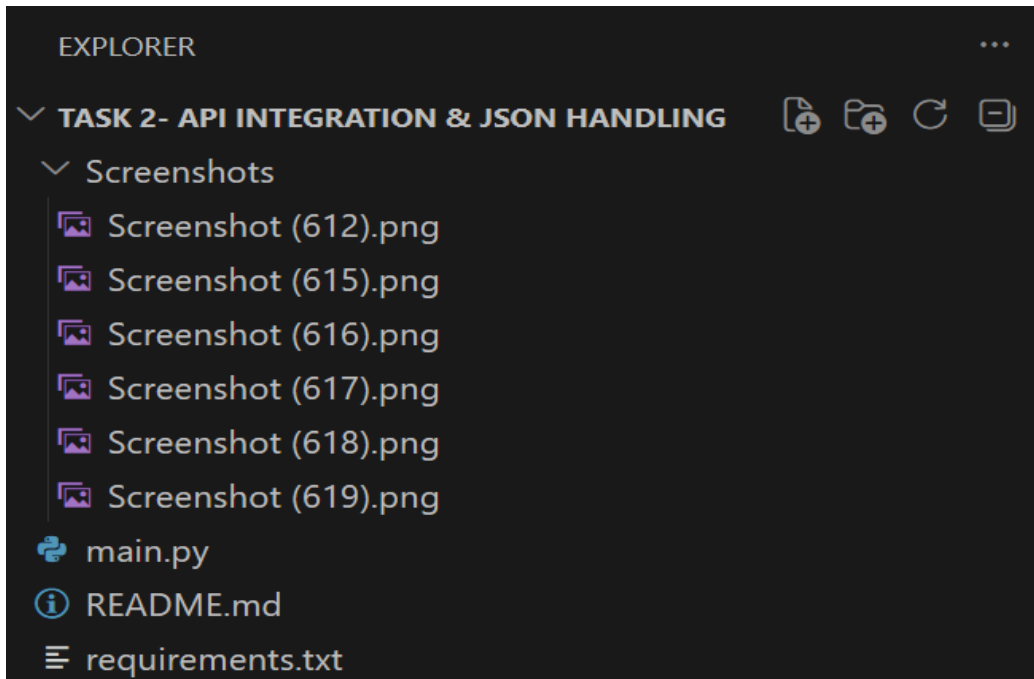
def main():
    print("\n☀️ WEATHER INFORMATION FINDER ☁️")
    # User input
    city = input("Enter city name: ")
    # Fetch weather data
    weather_data = get_weather(city)
    # Check if data exists
    if weather_data:
        display_weather(weather_data)
    else:
        print("Could not fetch weather data.")

# Run program
if __name__ == "__main__":
    main()

```

6. Output Screenshots

- PROJECT FOLDER STRUCTURE



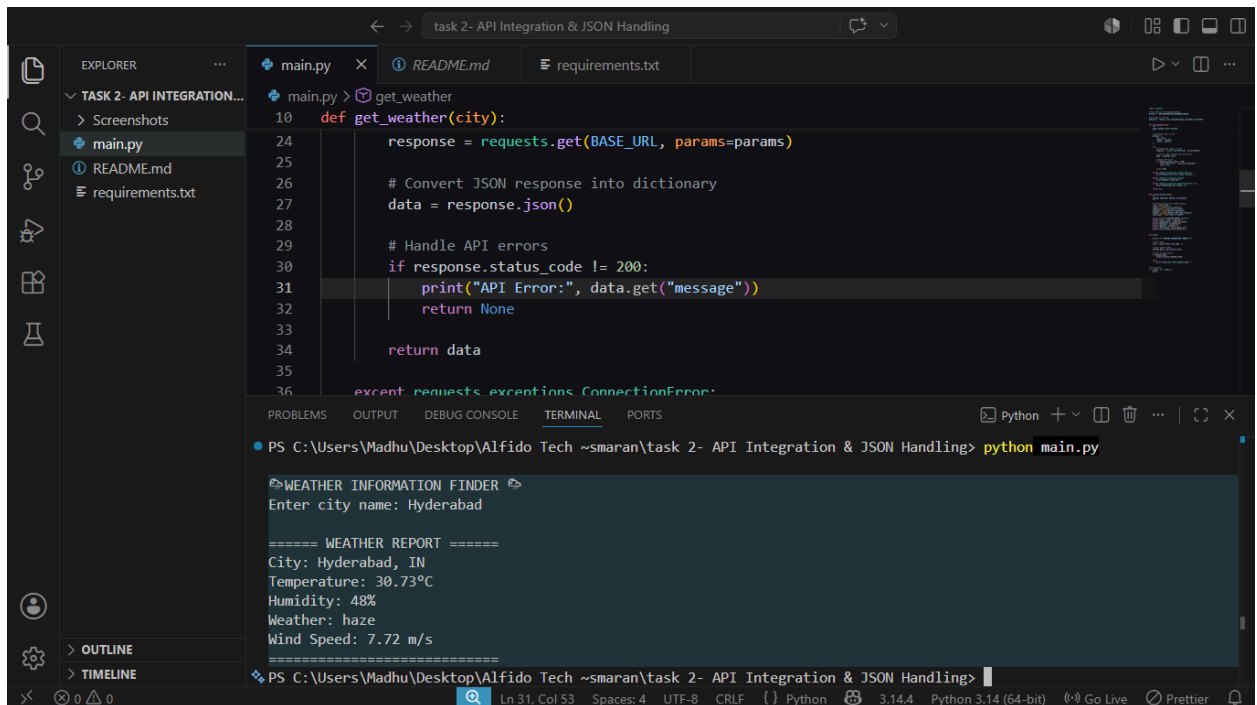
- API REQUEST CODE

```
# Parameters sent to API
params = {
    "q": city,
    "appid": API_KEY,
    "units": "metric"
}

try:
    # Sending GET request to API
    response = requests.get(BASE_URL, params=params)

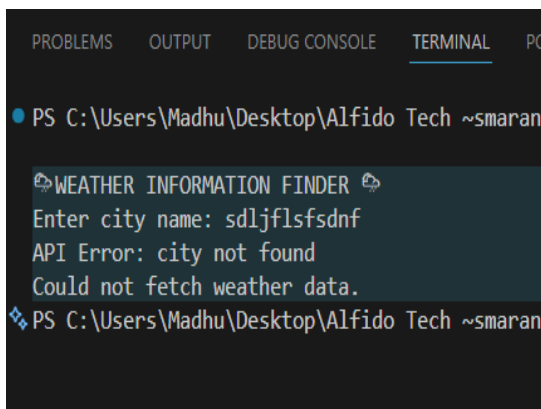
    # Convert JSON response into dictionary
    data = response.json()
```

- SUCCESSFUL OUTPUT

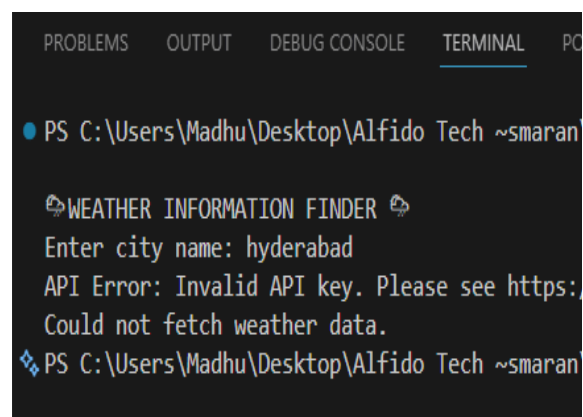


The screenshot shows a VS Code editor with a file explorer on the left containing 'TASK 2- API INTEGRATION...', 'Screenshots', 'main.py', 'README.md', and 'requirements.txt'. The main editor displays the code for 'main.py', specifically the 'get_weather' function. The terminal at the bottom shows the command 'python main.py' being executed, followed by the program's output: 'WEATHER INFORMATION FINDER', 'Enter city name: Hyderabad', and a detailed weather report for Hyderabad, IN, including temperature (30.73°C), humidity (48%), weather (haze), and wind speed (7.72 m/s).

- ERROR HANDLING OUTPUT



This screenshot shows the terminal output when an invalid city name 'sdljflsfsdnf' is entered. The program prints 'API Error: city not found' and 'Could not fetch weather data.'.



This screenshot shows the terminal output when an invalid API key is used. The program prints 'API Error: Invalid API key. Please see https:' and 'Could not fetch weather data.'.

7. Conclusion

This project helped in understanding API Integration, JSON handling, HTTP requests, real-time data fetching, and exception handling using Python.

8. GitHub Repo link:

<https://github.com/smaran-243/Alfido-Tech-Internship>

9. Linked In:

[API Integration & JSON Handling using Python](#)